

Docker

Course Summary – Docker (Pages 1–55)

1 Virtualization vs Containerization

♦ Virtualization

- Creates **virtual machines (VMs)**.
- Each VM has:
 - Its **own OS**
 - Its own libraries and dependencies
- Managed by a **hypervisor** (e.g. VirtualBox, VMware).
- Heavy on resources (CPU, RAM, disk).
- Slow startup (minutes).
- Strong isolation (security advantage).

 **Exam keywords:** hypervisor, full OS, heavy, slow, strong isolation

Use cases of virtualization

- Server consolidation
- Legacy applications

♦ Containerization

- Lightweight OS-level virtualization.
- Packages:
 - Application
 - Dependencies
 - Configuration
- Containers **share the host OS kernel**.
- Each container runs in **isolated user space**.
- Fast startup (seconds).
- Uses **Docker**.

 **Exam keywords:** shared kernel, lightweight, fast, isolated

Use cases of containerization

- Microservice architecture
- Container orchestration

✓ Key Difference (VERY IMPORTANT)

Containers are faster and lighter than virtual machines because they share the host OS kernel.

- **Docker vs VM:**

Containers are lightweight and share the OS kernel; VMs include a full OS.

2 Why Containers? (Development Problem)

✗ Before Containers

- Every developer installs:
 - Database
 - Backend
 - Frontend
 - Dependencies
- Different OS → different problems.
- Many steps → many errors.
- “Works on my machine” problem.

✓ With Containers

- Each service runs in **its own container**.
- Same command on **Windows / Linux / macOS**.
- One command to start a service:

`docker run postgres`

📌 Exam sentence:

Docker standardizes the development and deployment process across all environments.

Recap: Virtualization vs. containerization

Virtualization

- Creates a virtual version of a computing resource
- Full virtualization
- VM: Simulated computer system inside another computer

Containerization

- Packages application and dependencies into isolated environment
- OS-level virtualization
- Container: Isolated application environment

Benefits of containers vs. virtual machines

Factor	Container	Virtual machine	Physical machine
Isolation	**	***	*
Security	**	***	*
Space	***	**	*
Scalability	***	**	*
Costs	***	**	*
Flexibility	**	***	*

3 What is Docker?

Docker is:

- A **containerization platform**
- A tool to:
 - Build images
 - Run containers
 - Ship applications

Docker packages:

- Application code
- Runtime
- System tools
- Libraries
- Configuration

➔ Result: **one standardized unit**

4 Docker Architecture

Overview of Docker components

- Most important Docker components:
 - Docker Desktop
 - Docker Engine
 - Docker Client
 - Docker Daemon
 - Docker Objects
 - Docker Images
 - Docker Containers
 - Docker Registries

♦ Docker Client

- CLI (`docker run`, `docker build`, etc.)
- Used by developers.
- Sends commands to Docker Daemon.

♦ Docker Daemon (dockerd)

- Core Docker service.
- Manages:
 - Images
 - Containers
 - Volumes
 - Networks
- Listens to Docker API requests.

♦ Docker Host

- Machine where Docker runs.
- Contains:
 - Docker daemon
 - Images
 - Containers

- Networks
- Volumes

◆ Docker Registry

- Stores Docker images.
- Example:
 - **Docker Hub** (public)
 - Private registries
- Commands:

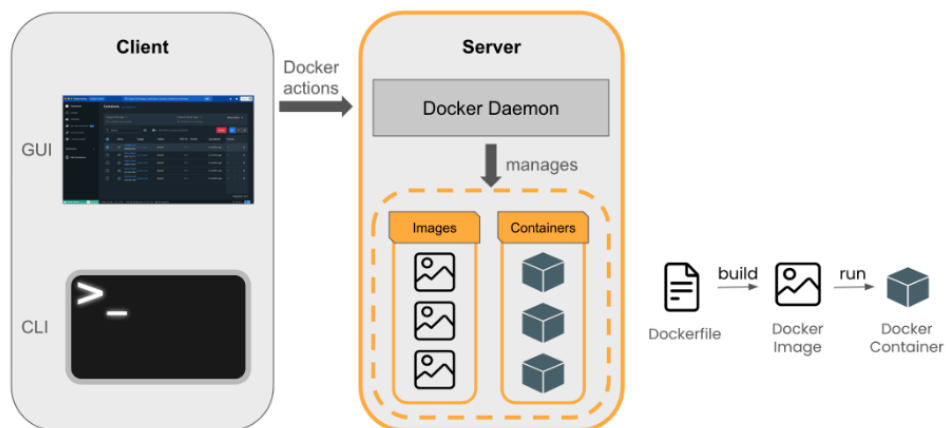
`docker pull`

`docker push`

📌 **Exam tip:** Docker Hub is a **Docker Registry**

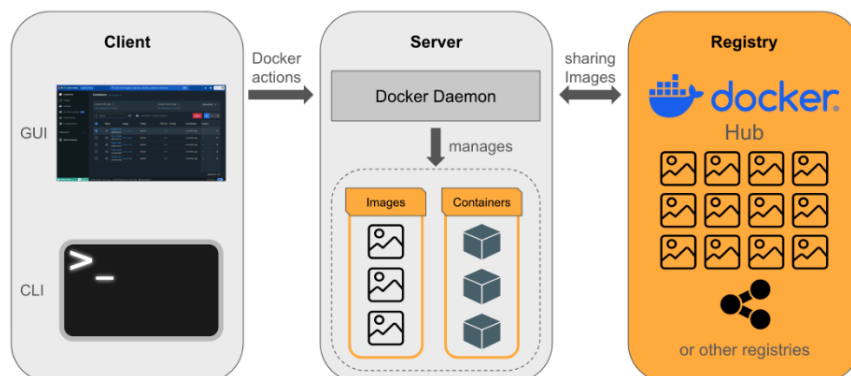
Overview of Docker objects

DOCKER ARCHITECTURE



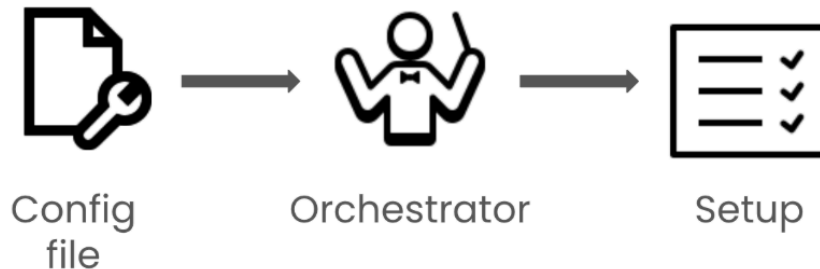
Sharing containers via registries

DOCKER ARCHITECTURE



Declarative programming in container orchestration

- Declarative programming:
 - Defining the desired output instead of describing the steps to reach it



5 Linux Isolation: Namespaces & Cgroups

◆ Namespaces (What you can SEE)

- Isolate resources per container.
- Each container thinks it has its own:
 - PID (processes)
 - Network
 - Users

→ Isolation of visibility

◆ Cgroups (What you can USE)

- Limit resources:
 - CPU
 - Memory
 - Disk
 - Network
- Prevents containers from consuming all resources.

→ Isolation of usage

📌 Very important exam sentence:

Namespaces limit what a container can see, cgroups limit how much it can use.

6 Docker Images vs Containers

◆ Docker Image

- Read-only
- Blueprint / template
- Contains:
 - App
 - Dependencies

- OS layer
 - Built using a **Dockerfile**
-

♦ Docker Container

- Running instance of an image
- Executes the application
- You can run **multiple containers from one image**

📌 Exam analogy:

- Image = Recipe 📖
- Container = Cooked meal 🍽️

📌 Image vs Container:

An image is a template, a container is a running instance.

7 Basic Docker Commands (VERY IMPORTANT)

List images

```
docker images
```

```
docker image ls
```

List containers

```
docker ps          # running
```

```
docker ps -a       # all
```

Docker info

```
docker info
```

8 docker run (CORE EXAM PART)

♦ Syntax

```
docker run [options] <image> <command>
```

⚠ Order matters:

options → image → command

♦ Important Options

-d (detached mode)

- Runs container in background

```
docker run -d nginx
```

-p host_port:container_port

- Port binding
- Allows access from host/browser

```
docker run -p 8080:80 nginx
```

-v host:container

- Volume mounting
- Data persistence

```
docker run -v /data:/var/lib/mysql mysql
```

--rm

- Deletes container after stop

```
docker run --rm ubuntu echo hello
```

-it

- Interactive terminal

```
docker run -it ubuntu /bin/bash
```

--name

- Custom container name

```
docker run --name my-nginx nginx
```

Important Notes

- No command → container stops immediately
- Avoid port & name collisions
- Containers are immutable

Option / Pattern	Meaning (simple)	Example	Key exam note / trap
<code>docker run [options] <image> <command></code>	Run a container from an image	<code>docker run ubuntu echo hi</code>	Order matters: options → image → command
<code>-d</code>	Run in background	<code>docker run -d nginx</code>	Use <code>docker ps</code> to see it running
<code>-p host:container</code>	Bind host port to container port	<code>docker run -p 8080:80 nginx</code>	Avoid port collision (same host port)
<code>-v host:container</code>	Mount host folder into container	<code>docker run -v /data:/var/lib/mysql mysql</code>	Data persists on host path
<code>--rm</code>	Delete container after it stops	<code>docker run --rm ubuntu echo hello</code>	Container won't be in <code>docker ps -a</code> after stop
<code>-it</code>	Interactive terminal	<code>docker run -it ubuntu /bin/bash</code>	Used for debugging/shell
<code>--name <name></code>	Custom container name	<code>docker run --name my-nginx nginx</code>	Avoid name collision (must be unique)
(No extra command)	Uses image default CMD/ENTRYPOINT	<code>docker run nginx</code>	If main process ends fast → container stops

9 Container Lifecycle

States:

- Created
- Running
- Paused
- Stopped
- Deleted

Commands:

`docker stop`

`docker start`

`docker rm`

10 Container Networking

♦ Bridge (default)

- Internal Docker network
 - Containers communicate using internal IP
 - Uses `docker0` bridge
-

♦ Host Network

- No isolation
 - Container uses host network directly
 - No port mapping needed
-

♦ None Network

- Completely isolated
 - No external or internal access
-

♦ User-Defined Networks

- Best practice
- Built-in DNS
- Containers communicate using **names**

`mysql.connect("mysql")`

Mode	What it is	Host access to container app	Container ↔ container communication	Best use / exam note	
Bridge (default)	Private Docker network (<code>docker0</code>), container gets private IP	✓ Needs <code>-p</code> (port mapping)	✓ Yes (same bridge network)	Default mode. Exam: without <code>-p</code> , browser can't reach it	
Host	Container uses host network directly (no isolation)	✓ No <code>-p</code> needed	⚠ Yes, but shares host networking	Exam trap: less isolation / more risky	
None	No network at all	✗ Not accessible	✗ No communication	Fully isolated (offline container)	
User-defined network	Private network like bridge + built-in DNS	✓ Needs <code>-p</code> only if host must access	✓ Yes, by name (DNS)	Best practice for multi-container apps: connect using <code>mysql</code> not IP	

11 Docker Volumes (VERY IMPORTANT)

✗ Problem

- Container data is lost when container stops.

✓ Solution: Docker Volumes

- Persistent data storage
- Stored outside container filesystem

Types of Volumes

Anonymous Volume

```
docker run -v /var/lib/mysql/data mysql
```

Named Volume

```
docker run -v myvol:/var/lib/mysql/data mysql
```

Create volume:

```
docker volume create myvol
```

📌 Used mainly for databases

8) Volumes (VERY IMPORTANT)

Problem

If you store data inside container filesystem, deleting container = data lost.

Solution: Volumes (persist outside container)

Two common patterns:

Anonymous volume

bash

Copier le code

```
docker run -v /var/lib/mysql/data mysql
```

Docker creates a volume automatically, name is random.

Named volume (best)

bash

Copier le code

```
docker volume create myvol
docker run -v myvol:/var/lib/mysql/data mysql
```

- ✓ Stable, reusable, perfect for databases.

12 Docker Compose (Pages ~53–55)

What is Docker Compose?

- Tool to manage **multi-container applications**
- Uses `docker-compose.yml`
- Runs services together

Common Commands

```
docker-compose up
docker-compose up -d
docker-compose down
docker-compose build
```

+ Docker Compose Commands

Docker Compose manages **multi-container applications** using:

`docker-compose.yml`

It starts, stops, rebuilds, and monitors **all services together**.

♦ `docker-compose up`

- Starts all services
- Creates containers, networks, volumes
- Shows aggregated logs
- `-d` → run in background

```
docker-compose up
docker-compose up -d
```

🔥 **Exam:** `up` does NOT rebuild images unless they don't exist.

♦ `docker-compose build`

- Rebuilds images from Dockerfiles
- Use after changing a Dockerfile or dependencies

```
docker-compose build
docker-compose up --build
```

♦ `docker-compose ps`

- Shows container status (running/stopped, ports)

```
docker-compose ps
```

♦ **docker-compose run**

- Runs a **one-time command**
- Starts dependent services automatically

```
docker-compose run web bash
```

```
docker-compose run --no-deps web bash
```

📌 **Use:** tests, migrations, debugging

♦ **docker-compose logs**

- Displays aggregated service logs

```
docker-compose logs
```

```
docker-compose logs web
```

♦ **docker-compose stop**

- Stops containers
- Does NOT remove anything

```
docker-compose stop
```

♦ **docker-compose rm**

- Removes stopped containers
- **-v** removes volumes (**data loss**)

```
docker-compose rm
```

```
docker-compose rm -v
```

♦ **docker-compose down**

- Removes containers + networks
- Volumes kept by default
- **-v** removes volumes

```
docker-compose down
```

```
docker-compose down -v
```

📌 **Exam:** resets the environment

🔑 **One-Line Exam Answers**

- **up** → starts services
- **build** → rebuilds images
- **ps** → shows status
- **run** → one-time command
- **logs** → logs

- **stop** → stops containers
 - **rm** → removes containers
 - **down** → destroys containers & networks
-

Why Docker Compose?

- Microservices
 - Single network
 - Easy startup & shutdown
 - Service-to-service communication
-



FINAL EXAM MEMORY BLOCK

- Docker ≠ VM (shared kernel)
- Image ≠ Container (template vs running)
- **docker run** options are critical
- Volumes = persistence
- Bridge network = default
- Docker Compose = multi-container apps

Questions

Docker (Containers) vs Virtual Machines (VMs)

Virtual Machines (VMs)

- Each VM has its **own operating system**.
- Runs on a **hypervisor**.
- Uses **more CPU, memory, and disk**.
- **Slower** to start (minutes).
- **Stronger isolation** (separate OS).

Docker Containers

- Containers **share the host OS kernel**.
- No full OS inside each container.
- **Lightweight** and efficient.
- **Very fast** startup (seconds).
- Isolation is weaker than VMs.

Exam Comparison Table

Feature	Virtual Machine	Docker Container
OS	Full OS per VM	Shared host OS
Startup time	Slow	Fast
Resource usage	High	Low
Size	Large (GBs)	Small (MBs)
Scalability	Slower	Very fast
Isolation	Strong	Weaker

 **Exam sentence to remember:**

Containers are lightweight and faster because they share the host OS kernel, while virtual machines run a full operating system.

2 Docker Image vs Docker Container

Docker Image

- A **blueprint / template**.
- Read-only.
- Contains:
 - Application code
 - Dependencies
 - OS libraries
- Built using a **Dockerfile**.
- Does **not run** by itself.

Docker Container

- A **running instance** of an image.
- Created from an image.
- Executes the application.
- You can run **many containers from one image**.

🔑 Simple Analogy (Very good for exams)

- **Image** = Recipe 📖
- **Container** = Cooked meal 🍽️

🔑 Exam Comparison Table

Feature	Docker Image	Docker Container
---------	--------------	------------------

State	Static	Running
Purpose	Template	Execution
Editable	No	Yes (runtime)
Quantity	One image	Multiple containers

➡ **Exam sentence to remember:**

A Docker image is a read-only template, while a Docker container is a running instance of that image.

Ultra-Short Exam Answers (Memorize)

- **Docker vs VM:**
Containers are lightweight and share the OS kernel; VMs include a full OS.
- **Image vs Container:**
An image is a template, a container is a running instance.

K8S



Kubernetes (Pages 56–135) — VERY DETAILED EXAM SUMMARY

1 What is Kubernetes? (Page 56–59)

Kubernetes (K8s) is:

- An **open-source container orchestration platform**
- Developed by **Google**
- Used to manage **containerized applications** across:
 - Physical machines
 - Virtual machines
 - Cloud environments

The name **Kubernetes** comes from Greek, meaning “**helmsman / pilot**”.

K8s = K + 8 letters + s.

Why Kubernetes exists

- Containers increased massively
- Shift from **monolith** → **microservices**
- Managing hundreds of containers manually is impossible

📌 **Exam sentence:**

Kubernetes automates deployment, scaling, and management of containerized applications.

2 Need for Container Orchestration (Page 60–61)

Problems without orchestration:

- Many containers
- Many services
- Manual scripts become complex
- No automatic recovery

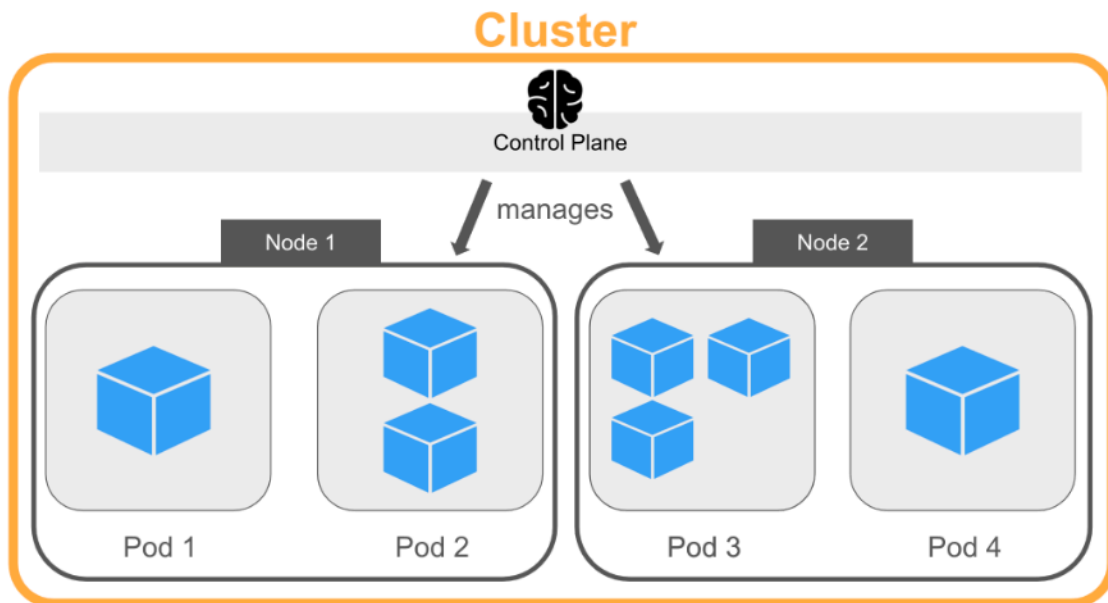
Orchestration tools provide:

- **Scalability** (scale up/down)
- **High availability** (no downtime)
- **Disaster recovery** (restart failed containers)

📌 **Exam keyword:** orchestration = automation + management

3 Kubernetes Cluster Architecture (Page 62–65)

A **Kubernetes cluster** is made of:



◆ Control Plane (Master Node)

Controls and manages the cluster.

Components:

1 API Server

- Entry point of the cluster
- Receives all commands:
 - CLI (`kubectl`)
 - UI
 - API

📌 **Exam:** all communication goes through API Server

2 Scheduler

- Decides **where Pods run**
 - Chooses the best node based on:
 - CPU
 - Memory
 - Current workload
-

3 Controller Manager

- Watches cluster state
 - Fixes problems automatically
 - Enables **self-healing**
-

4 etcd

- Key-value database
- Stores:
 - Cluster state
 - Configuration
- Very critical component

📌 **Exam sentence:**

etcd is the backing store of Kubernetes.

♦ Worker Nodes

Where applications run.

Each worker node contains:

- **kubelet** → communicates with control plane
- Container runtime (Docker)
- Pods running application containers

📌 **Exam:**

Control Plane manages, Worker Nodes execute.

4 Virtual Network (Page 64)

- Connects all nodes together
 - Makes the cluster act like **one powerful machine**
 - Enables Pod-to-Pod communication
-

5 Main Kubernetes Components (Page 66)

Core objects:

- Pods
 - ReplicaSets
 - Deployments
 - Services
 - ConfigMaps
 - Secrets
 - Namespaces
-

6 Pods (Page 67–70) ★ VERY IMPORTANT

What is a Pod?

- Smallest deployable unit in Kubernetes
- Kubernetes does **NOT** deploy containers directly
- A Pod wraps:
 - One container (most common)
 - Or multiple containers

Characteristics:

- One IP address
- Shared network
- Shared storage
- Single application instance

📌 **Exam sentence:**

A Pod is the smallest unit of deployment in Kubernetes.

♦ **Multi-Container Pods**

- Multiple containers inside one Pod
 - Share lifecycle and network
 - Often used with:
 - Main container
 - Helper / sidecar container
-

7 Managing Pods with kubectl (Page 71–72)

Create a Pod

- `kubectl run mypod --image=nginx`

Create Pod with port

- `kubectl run mypod --image=nginx --port=80`

Interactive Pod

- `kubectl run -it testpod --image=ubuntu -- /bin/bash`

List Pods

- `kubectl get pods`

Pod details

- `kubectl describe pod mypod`

Logs

- `kubectl logs mypod`

Delete Pod

- `kubectl delete pod mypod`
-

8 YAML in Kubernetes (Page 73)

Kubernetes is **declarative**.

Main YAML fields:

- `apiVersion`
- `kind`
- `metadata`
- `spec`

Create objects:

- `kubectl apply -f file.yml`

 **Exam:** YAML defines desired state.

9 Commands & Arguments (Page 74)

Kubernetes vs Docker mapping:

- Docker `CMD` / `ENTRYPOINT`
- Kubernetes `command` / `args`

Assembling instructions to Dockerfile

```
# Define the image on which to build
FROM python:3.10

# Copy files/folders to the main folder of the container
COPY . .

# Install the application's dependencies
RUN pip install -r requirements.txt

# Run the script when the container starts
ENTRYPOINT ["python", "hello_world.py"]
```

10 Environment Variables (Page 75–76)

Ways to define environment variables:

1. Plain key-value
 2. ConfigMaps
 3. Secrets
-

11 ConfigMaps (Page 77–81)

What is a ConfigMap?

- Stores **non-sensitive** configuration
- Key-value pairs
- Injected into Pods

Creation:

- Imperative
- Declarative (YAML)

 **Exam sentence:**

ConfigMaps decouple configuration from application code.

12 Secrets (Page 82–87)

What is a Secret?

- Stores **sensitive data**
 - Passwords
 - Tokens
 - Keys
- Base64 encoded
- Safer than plain text

Injection methods:

- Environment variables
- Volumes

 **Exam sentence:**

Secrets store confidential data securely.

13 Resource Management (Page 88–89)

Requests

- Minimum guaranteed resources
- Used by scheduler

Limits

- Maximum allowed usage

Behavior:

- Memory limit exceeded → **OOMKilled**
- CPU limit exceeded → **throttled**

 **Exam sentence:**

Requests guarantee resources, limits restrict usage.

14 Observability – Probes (Page 90–94)

Readiness Probe

- Is the app ready to receive traffic?

Liveness Probe

- Is the app alive?

Probe types:

- HTTP
- TCP
- Command

 **Exam:**

Readiness controls traffic, Liveness controls restarts.

15 ReplicaSets (Page 95–97)

- Ensures a **fixed number of Pods**
- Automatically replaces failed Pods

 **Exam sentence:**

ReplicaSets maintain the desired number of Pods.

16 Deployments (Page 98)

- Higher-level abstraction
- Manages ReplicaSets
- Enables:
 - Updates
 - Rollbacks

 **Exam sentence:**

Deployments manage application lifecycle.

17 Services (Page 99–106)

What is a Service?

- Exposes Pods
- Provides stable access

Service Types:

ClusterIP

- Internal communication only

NodePort

- Exposes app on node IP + port

LoadBalancer

- Cloud load balancer

 **Exam sentence:**

Services enable communication between Pods.

18 Namespaces (Page 107–111)

Namespaces provide **logical isolation**.

Default namespaces:

- default
- kube-system
- kube-public
- kube-node-lease

 **Exam sentence:**

Namespaces isolate resources inside a cluster.

19 Deployment Strategies (Page 112–130)

Recreate

- Old Pods deleted first
- Downtime

Rolling Update (default)

- Gradual replacement
- No downtime

Blue/Green

- Two environments
- Switch traffic

Canary

- Small percentage of traffic to new version

Exam:

Rolling Update provides zero downtime deployments.

20 Jobs & CronJobs (Page 131–135)

Job

- Runs tasks until completion
- Retries on failure

CronJob

- Scheduled Jobs
- Like Linux cron

Exam sentence:

CronJobs execute Jobs on a schedule.



FINAL KUBERNETES EXAM BLOCK (MEMORIZE)

- Pod = smallest unit
- Deployment manages Pods
- Service exposes Pods
- ConfigMap = config
- Secret = sensitive data
- Requests vs Limits
- Rolling Update = no downtime
- Job = one-time task
- CronJob = scheduled task

Onglet 4

True	False
Virtual machines offer greater scalability and flexibility than physical machines.	Applications running in different virtual machines are less isolated than applications running on the same physical machine.
Virtualization maximizes resource utilization, resulting in cost efficiencies and sustainable usage.	Virtual machines typically cause more downtime than physical machines.
VMs are platform-independent, and different operating systems can run seamlessly on the same host machine.	

True	False
Containers start up quickly because there is no need to load the entire operating system.	Applications can run on different operating systems on the same host machine because containerization is virtualization at the OS-level.
Containers on the same host are isolated, so a crash in one container will not cause a crash in another.	Applications with different hardware requirements can run on the same host machine because the hardware can be customized for each container.
Applications are easily portable and reproducible in different environments because containers contain everything needed to run the application.	

(false loula !)

True	False
Container orchestration can improve application performance by increasing reliability and reducing downtime.	Container orchestration offers more customization than the manual management of containers.
Container orchestration can reduce infrastructure costs for large-scale applications.	Container orchestration saves time when applied to small-scale applications.
Container orchestration makes it easy to scale applications on demand.	
Automation through container orchestration increases developer productivity.	

(zouz el false!)

True	False
The <code>FROM</code> instruction defines the image on which the new image is to be built.	The <code>ENTRYPOINT</code> defines the image you are building on.
The <code>ENTRYPOINT</code> instruction defines the main command of the container, which serves as its main purpose.	The <code>RUN</code> instruction runs the Docker container.
The <code>RUN</code> instruction runs a command inside the container.	
The <code>COPY</code> instruction copies files or directories into the container.	